Créer un site web de présentation

Guide complet de la création des fichiers à la mise en ligne sur GitHub Pages

Ce guide vous accompagne pas à pas dans la création d'un site web statique pour présenter un projet (portfolio, projet scolaire, projet open-source…). Vous n'avez besoin que d'un éditeur de texte, d'un navigateur et d'un compte GitHub — **aucun serveur payant requis**.

■ Sommaire

1	Prérequis & outils
2	Structure du projet
3	Écriture du fichier index.html
4	Mise en forme avec style.css
5	Tester en local
6	Créer un dépôt GitHub
7	Envoyer les fichiers (push)
8	Activer GitHub Pages
9	Personnaliser & aller plus loin

ÉTAPE 1 Prérequis & outils nécessaires

Avant de commencer, assurez-vous de disposer des éléments suivants :

Éditeur de code	Visual Studio Code (recommandé), Sublime Text, Notepad++…
Navigateur web	Chrome, Firefox ou Edge pour tester votre site en local.
Compte GitHub	Gratuit sur **github.com** — nécessaire pour l'hébergement.
Git (optionnel)	Pour la ligne de commande. Sinon, l'interface GitHub.com suffit.

■ VS Code est gratuit et disponible sur code.visualstudio.com. Installez l'extension **Live Server** pour prévisualiser vos fichiers.

ÉTAPE 2 Structure du projet

Créez un **dossier dédié** sur votre ordinateur (ex. *mon-projet/*). À l'intérieur, vous n'avez besoin que de deux fichiers essentiels :

```
Structure recommandée

mon-projet/

■■■ index.html      ← page principale du site
```

```
■■■ style.css        ← feuille de style (mise en forme)
■■■ assets/          ← (optionnel) images, icônes…
    ■■■ logo.png
```

Le fichier **index.html** est toujours la page d'accueil par défaut — les serveurs web (y compris GitHub Pages) le chargent automatiquement.

ÉTAPE 3 Écriture du fichier index.html

Ouvrez votre éditeur et créez **index.html**. Voici un squelette complet avec les sections clés d'une page de présentation :

index.html — structure de base

```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mon Projet</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

  <!-- En-tête -->
  <header>
    <h1>Nom du Projet</h1>
    <p class="tagline">Une courte description accrocheuse.</p>
  </header>

  <!-- Navigation -->
  <nav>
    <a href="#apropos">À propos</a>
    <a href="#fonctionnalites">Fonctionnalités</a>
    <a href="#contact">Contact</a>
  </nav>

  <main>
    <section id="apropos">
      <h2>À propos</h2>
      <p>Décrivez votre projet ici.</p>
    </section>

    <section id="fonctionnalites">
      <h2>Fonctionnalités</h2>
```

```
      <ul>
        <li>Fonctionnalité 1</li>
        <li>Fonctionnalité 2</li>
      </ul>
    </section>

    <section id="contact">
      <h2>Contact</h2>
      <p>Email : votre@email.com</p>
    </section>
  </main>

  <footer>
    <p>&copy; 2025 — Mon Projet</p>
  </footer>

</body>
</html>
```

Balises HTML essentielles à connaître :

`<h1> … <h6>`	Titres (h1 = plus important, h6 = moins important)
`<p>`	Paragraphe de texte
`<a href="…">`	Lien hypertexte
`<img src="…">`	Image
`<ul> / <li>`	Liste à puces
`<div>`	Conteneur générique (mise en page)
`<section>`	Section thématique de la page
`<header> / <footer>`	En-tête / pied de page

ÉTAPE 4 Mise en forme avec style.css

Créez **style.css** dans le même dossier. La règle de base : *sélecteur { propriété: valeur; }*

style.css — exemple complet

```
/* Réinitialisation de base */
* { margin: 0; padding: 0; box-sizing: border-box; }

body {
  font-family: 'Segoe UI', Arial, sans-serif;
  background-color: #f5f5f5;
  color: #333;
  line-height: 1.6;
```

```
}

/* En-tête */
header {
  background: linear-gradient(135deg, #1a1a2e, #0f3460);
  color: white;
  text-align: center;
  padding: 60px 20px;
}
header h1 { font-size: 2.5rem; margin-bottom: 10px; }
.tagline   { font-size: 1.1rem; opacity: 0.85; }

/* Navigation */
nav {
  display: flex;
  justify-content: center;
  gap: 30px;
  background: #0f3460;
  padding: 12px;
}
nav a { color: white; text-decoration: none; font-weight: bold; }
nav a:hover { color: #e94560; }

/* Sections */
section {
  max-width: 900px;
  margin: 40px auto;
  padding: 30px;
  background: white;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}
section h2 { color: #0f3460; margin-bottom: 15px; }

/* Pied de page */
footer {
  text-align: center;
  padding: 20px;
  background: #1a1a2e;
  color: #aaa;
  font-size: 0.9rem;
```

```
}
```

Propriétés CSS à connaître :

`color`	Couleur du texte (ex: #333, red, rgb(0,0,0))
`background-color`	Couleur de fond
`font-size`	Taille du texte (ex: 16px, 1.2rem)
`margin / padding`	Espace extérieur / intérieur d'un élément
`display: flex`	Mise en page flexbox (alignement facile)
`border-radius`	Coins arrondis
`box-shadow`	Ombre portée autour d'un élément
`max-width`	Largeur maximale (ex: 900px)

ÉTAPE 5 Tester votre site en local

Avant de mettre en ligne, testez toujours votre site sur votre machine :

- Double-cliquez sur **index.html** → il s'ouvre dans votre navigateur.
- Avec **VS Code + Live Server** : clic droit sur index.html → *Open with Live Server*. La page se recharge automatiquement à chaque sauvegarde.
- Utilisez la touche **F12** (Outils de développement) pour inspecter les éléments et déboguer le CSS en temps réel.

> ■ Vérifiez que le lien <link rel="stylesheet" href="style.css"> dans index.html pointe bien vers le bon fichier. La casse (majuscules/minuscules) compte !

ÉTAPE 6 Créer un dépôt GitHub

Un **dépôt (repository)** est l'espace en ligne où seront hébergés vos fichiers.

1. Connectez-vous à **github.com**.
2. Cliquez sur le bouton **« + »** en haut à droite → *New repository*.
3. Donnez un nom au dépôt (ex: *mon-projet-web*).
4. Laissez-le en **Public** (obligatoire pour GitHub Pages gratuit).
5. Cochez **« Add a README file »** (facultatif mais recommandé).
6. Cliquez sur **« Create repository »**.

ÉTAPE 7 Envoyer vos fichiers sur GitHub (push)

Deux méthodes pour uploader vos fichiers :

Méthode A — Via l'interface web GitHub (la plus simple)

- **1.** Ouvrez votre dépôt sur github.com.
- **2.** Cliquez sur **« Add file »** → *Upload files*.

- **3.** Glissez-déposez **index.html** et **style.css** dans la zone prévue.
- **4.** Ajoutez un message de commit (ex: *Ajout des fichiers du site*).
- **5.** Cliquez sur **« Commit changes »**.

Méthode B — Via Git en ligne de commande

```
Terminal / Invite de commandes

# Se placer dans le dossier du projet
cd chemin/vers/mon-projet

# Initialiser Git
git init
git remote add origin https://github.com/VOTRE_PSEUDO/mon-projet-web.git

# Ajouter, valider et envoyer
git add .
git commit -m "Premier commit : ajout du site"
git push -u origin main
```

■ Remplacez VOTRE_PSEUDO par votre nom d'utilisateur GitHub et mon-projet-web par le nom exact de votre dépôt.

ÉTAPE 8 Activer GitHub Pages

GitHub Pages publie votre site gratuitement depuis votre dépôt :

1. Dans votre dépôt, cliquez sur l'onglet **« Settings »**.
2. Dans le menu gauche, allez dans **« Pages »**.
3. Sous *Source*, sélectionnez la branche **main** et le dossier **/ (root)**.
4. Cliquez sur **« Save »**.
5. Attendez 1 à 2 minutes, puis rafraîchissez la page.
6. Votre URL apparaît sous la forme : **https://VOTRE_PSEUDO.github.io/mon-projet-web/**

■ ■ Votre site est maintenant en ligne et accessible par n'importe qui via l'URL GitHub Pages ! Partagez-la librement.

■ Pour mettre à jour le site, il suffit de modifier vos fichiers et de les envoyer à nouveau sur GitHub (upload ou git push). GitHub Pages se met à jour automatiquement en quelques secondes.

ÉTAPE 9 Personnaliser & aller plus loin

■ ■ **Ajouter des images**	Placez vos images dans assets/ et référencez-les : <img src="assets/logo.png" alt="Logo">
■ **Rendre le site responsive**	Utilisez @media (max-width: 768px) en CSS pour adapter l'affichage sur mobile.

■ **Google Fonts**	Ajoutez une <link> Google Fonts dans <head> pour des polices personnalisées.
■ **Animations CSS**	Utilisez transition, animation et @keyframes pour des effets visuels.
■ **Nom de domaine personnalisé**	Dans Settings > Pages, ajoutez votre propre domaine (ex: monprojet.fr).
■ **Plusieurs pages**	Créez d'autres fichiers .html (ex: apropos.html) et liez-les avec <a href="apropos.html">.

■ Ressources utiles

MDN Web Docs	developer.mozilla.org — référence HTML & CSS complète
W3Schools	w3schools.com — tutoriels interactifs pour débutants
GitHub Docs	docs.github.com/pages — documentation officielle GitHub Pages
CSS Tricks	css-tricks.com — guides CSS avancés et exemples
Can I Use	caniuse.com — vérifier la compatibilité des propriétés CSS